

UNIVERSIDAD TECNOLÓGICA DE BAHÍA DE BANDERAS



Materia:

Analítica de Datos Para Negocios Digitales

Docente:

Guillermo Quiñones Villarreal

Actividad:

Reporte Escrito

Que presenta:

Brayan Paul Pelayo Ascencio

Danna Geraldine Gonzalez Gaviño

Carrera:

INGENIERÍA TECNOLOGÍAS DE LA INFORMACIÓN, ENTORNOS VIRTUALES Y NEGOCIOS DIGITALES

Nuevo Nayarit, Nayarit, 9 de Junio de 2026

Tabla de contenido

Introducción	3
1. Contexto del Negocio	4
2. Descripción del Proceso ETL	5
2.1 Extracción	5
2.2 Transformación	5
2.3 Validación	6
3. Evidencias	7
3.1 Preparación del entorno y archivos del proyecto	7
3.2 Ejecución de la herramienta CSV Scanner	8
3.3 Carga y análisis del dataset Global Superstore Sales	9
3.4 Resultados del análisis de calidad de datos	10
3.5 Generación de scripts y reportes	12
3.6 Revisión y corrección del script SQL	14
3.7 Ejecución de la extracción en PostgreSQL	16
3.8 Transformación hacia el esquema transformation	18
3.9 Validación final mediante pgAdmin	20
4. Base de Datos Procesada	22
4.1 Esquema staging	22
4.2 Esquema transformation	23
Conclusiones	24

Introducción

En el presente proyecto se llevó a cabo el proceso ETL (Extract, Transform and Load; Extracción, Transformación y Carga), una metodología utilizada para obtener datos desde una fuente de origen, transformarlos de acuerdo con las necesidades del negocio y cargarlos en una estructura organizada para su posterior análisis.

Para el desarrollo de esta práctica se utilizó el dataset ficticio Global Superstore Sales, publicado por Tableau como material didáctico para la enseñanza y práctica de técnicas de análisis de datos. La utilización de este conjunto de datos permitió trabajar con información representativa de un entorno empresarial real sin comprometer información sensible o confidencial.

El presente reporte recapitula el proceso completo realizado durante la práctica, comenzando con la extracción de los datos originales contenidos en el dataset, continuando con su análisis y limpieza mediante el uso de la herramienta desarrollada [CSV Scanner](#), así como la realización de ajustes y correcciones manuales cuando fue necesario. Posteriormente, se llevaron a cabo procesos de validación para verificar la integridad, consistencia y calidad de la información obtenida, así como la correcta carga de los registros en la base de datos de destino.

Finalmente, se documentan los resultados obtenidos en cada una de las etapas del proceso ETL, incluyendo las validaciones realizadas sobre las tablas generadas y los datos procesados. Las evidencias gráficas que respaldan cada fase del desarrollo se presentan en este documento (y en un documento complementario de evidencias adjunto a este reporte).

Debido a las características particulares de la práctica, la etapa final se enfocó en la validación de los datos transformados, sustituyendo una fase de carga adicional independiente.

1. Contexto del Negocio

Para el desarrollo de esta práctica se utilizó el conjunto de datos **Global Superstore Sales**, un dataset ficticio publicado por **Tableau** con fines educativos y de capacitación en análisis de datos. Este conjunto de información simula las operaciones comerciales de una empresa multinacional dedicada a la venta de productos de diversas categorías en diferentes regiones del mundo.

El dataset contiene información relacionada con órdenes de compra, clientes, productos, categorías, segmentos de mercado, regiones geográficas, fechas de venta y envío, así como indicadores financieros asociados a cada transacción, tales como ventas, descuentos, costos de envío y ganancias obtenidas. Gracias a la diversidad de variables incluidas, el conjunto de datos permite realizar análisis comerciales, financieros y operativos similares a los que se desarrollan en entornos empresariales reales.

Desde una perspectiva de negocio, la información contenida en el dataset puede utilizarse para identificar patrones de consumo, evaluar el desempeño de productos y categorías, analizar la rentabilidad de las operaciones, comparar resultados entre regiones y apoyar la toma de decisiones estratégicas basadas en datos.

Sin embargo, antes de que la información pueda ser utilizada con fines analíticos, es necesario asegurar su calidad y consistencia. Los datos provenientes de fuentes externas suelen presentar problemas como formatos inconsistentes, tipos de datos incorrectos, valores nulos, registros duplicados o información que requiere transformación para adaptarse a la estructura de una base de datos analítica.

Por este motivo, se implementó un proceso ETL (Extracción, Transformación y Carga) cuyo objetivo fue preparar los datos del dataset para su posterior análisis, garantizando que la información almacenada en la base de datos final mantuviera criterios adecuados de integridad, consistencia y calidad.

2. Descripción del Proceso ETL

El proceso ETL desarrollado durante esta práctica tuvo como objetivo preparar la información contenida en el dataset **Global Superstore Sales** para su posterior análisis dentro de una base de datos estructurada. Para ello se siguieron las tres etapas fundamentales de la metodología ETL: extracción de los datos originales, transformación y limpieza de la información, y validación (en lugar de la carga por esta ocasión) de los datos procesados. En las siguientes secciones se describen detalladamente las actividades realizadas en cada una de estas fases, así como las herramientas y procedimientos utilizados para garantizar la calidad e integridad de los datos.

2.1 Extracción

La primera etapa del proceso ETL consistió en la obtención y carga inicial de los datos en una base de datos PostgreSQL sin realizar modificaciones sobre la información original. Para ello, se utilizó el dataset **Global Superstore Sales**, asignado para el desarrollo de la práctica y descargado desde la plataforma **Kaggle** en formato CSV.

Una vez obtenido el conjunto de datos, se investigaron diferentes alternativas para realizar la carga masiva de información hacia PostgreSQL. Después de evaluar distintas opciones, se determinó que el método más adecuado para este proyecto era la utilización del comando **\copy** de PostgreSQL ejecutado desde un archivo SQL mediante consola de comandos. Esta alternativa permitió importar grandes volúmenes de información de forma eficiente y reproducible, manteniendo la integridad de los datos originales.

Con base en esta decisión se desarrolló el archivo **01_extraccion.sql**, encargado de automatizar la creación de la base de datos de trabajo, la generación del esquema **staging** y la creación de la tabla **orders** destinada a almacenar la información del archivo CSV en su estado original.

Durante esta etapa se respetó el principio fundamental del área de staging: conservar los datos tal como fueron recibidos desde la fuente de origen. Por esta razón, todos los registros fueron cargados sin aplicar procesos de limpieza, transformación o validación, garantizando la disponibilidad de una copia íntegra del dataset para las etapas posteriores del proceso ETL.

Como resultado de esta fase se obtuvo una base de datos **super_store** con el esquema **staging** correctamente configurado y la tabla **orders** poblada con la información original del dataset, lista para ser utilizada en las etapas de transformación y análisis.

2.2 Transformación

La etapa de transformación representó la fase de mayor complejidad dentro del proyecto. Con el objetivo de facilitar el análisis, limpieza y preparación de los datos, se desarrolló una herramienta propia denominada [**CSV Scanner**](#), diseñada específicamente para analizar archivos CSV, identificar

posibles problemas de calidad de datos y generar automáticamente recursos de apoyo para el proceso ETL.

El desarrollo de la herramienta comenzó desde la asignación de la práctica y se realizó de manera paralela al trabajo sobre el dataset. A lo largo de su implementación se llevaron a cabo múltiples ciclos de prueba utilizando el conjunto de datos **Global Superstore Sales**, realizando ajustes continuos hasta alcanzar una versión completamente funcional y adecuada para las necesidades del proyecto.

Una vez completado el desarrollo de la herramienta, el archivo CSV fue sometido a diversos procesos de análisis orientados a identificar la estructura de los datos, los tipos de información presentes en cada columna, posibles inconsistencias de formato y oportunidades de mejora para la construcción de la base de datos analítica. Adicionalmente, la herramienta generó reportes de apoyo y scripts SQL que sirvieron como punto de partida para la fase de transformación.

Posteriormente, los resultados obtenidos fueron revisados manualmente con el fin de verificar su coherencia y corregir detalles específicos que requerían intervención humana. Entre las actividades realizadas se incluyeron la validación de nombres de campos, revisión de estructuras de tablas, ajustes en el script SQL generado y verificación de los entornos de ejecución necesarios para garantizar el correcto funcionamiento del proceso.

Asimismo, fue necesario realizar diversas pruebas de ejecución en PostgreSQL para validar la compatibilidad del script generado, corregir errores detectados durante la implementación y asegurar que la transformación de los datos produjera resultados consistentes. Este proceso iterativo permitió refinar progresivamente la solución hasta obtener una estructura de transformación estable y adecuada para la etapa de validación y análisis posterior.

Como resultado de esta fase se obtuvo el script necesario para transformar la información almacenada en el esquema de **staging** y prepararla para su incorporación al esquema **transformation** con sus respectivas tablas **datos_limpios** y **datos_cuarentena** (esta última fue diseñada para almacenar datos corruptos o registros nulos) dentro de la base de datos analítica **super_store**, garantizando la consistencia de los nombres de campos, la correcta interpretación de los tipos de datos y la integridad general de la información procesada.

2.3 Validación

Aunque la práctica se planteó bajo el enfoque general de un proceso ETL, en el desarrollo realizado la fase final no consistió en una carga adicional independiente, sino en la validación de la estructura resultante después del proceso de transformación. La información ya había sido procesada y almacenada en el esquema **transformation**, por lo que esta etapa se enfocó en comprobar que los datos transformados fueran consistentes, completos y adecuados para su posterior análisis.

Para ello se generó un script SQL de validación compuesto por diversas consultas **SELECT**, cuyo propósito fue revisar la integridad de las tablas resultantes **transformation.datos_limpios** y **transformation.datos_cuarentena**. Estas consultas permitieron verificar la existencia de las tablas,

el conteo de registros, la estructura de columnas, la presencia de valores nulos, posibles duplicados, consistencia de fechas, rangos numéricos y distribución de datos por categorías relevantes.

La ejecución de estas validaciones se realizó mediante la herramienta **Query Tool** de **pgAdmin**, utilizando PostgreSQL 18 como motor de base de datos. Este entorno permitió revisar de forma directa los resultados de cada consulta y comprobar que la transformación aplicada al dataset se hubiera realizado correctamente.

Como resultado de esta etapa, se confirmó que los datos almacenados en **transformation.datos_limpios** se encontraban correctamente estandarizados, incluyendo campos de fecha y valores numéricos como ventas, descuentos, costos de envío y ganancias. Asimismo, se verificó que no existían datos corruptos o registros inválidos que requirieran ser aislados en la tabla **transformation.datos_cuarentena**, lo cual indica que el proceso de transformación fue exitoso para el dataset utilizado.

En conclusión, esta fase permitió comprobar que la base de datos procesada cumplía con los criterios mínimos de calidad requeridos para ser utilizada en análisis posteriores, manteniendo trazabilidad mediante la tabla de cuarentena y asegurando que la información limpia estuviera disponible en la estructura analítica correspondiente.

3. Evidencias

En esta sección se presentan las evidencias obtenidas durante el desarrollo de la práctica, siguiendo el orden real en que se llevó a cabo el trabajo. Las capturas documentan desde la preparación del entorno y la extracción inicial de los datos, hasta el uso de la herramienta CSV Scanner, la generación y ajuste de scripts SQL, y la validación final de las tablas procesadas en PostgreSQL.

Las evidencias incluidas permiten comprobar que el proceso se desarrolló de forma progresiva, partiendo de los datos originales del dataset **Global Superstore Sales**, pasando por su análisis y transformación, y finalizando con la revisión de los resultados almacenados en los esquemas correspondientes de la base de datos.

3.1 Preparación del entorno y archivos del proyecto

En esta primera evidencia se muestra la organización inicial del proyecto, incluyendo los archivos SQL, el dataset utilizado y los elementos necesarios para ejecutar el proceso. Esta etapa permitió mantener una estructura ordenada para trabajar con los scripts de extracción, transformación y validación.

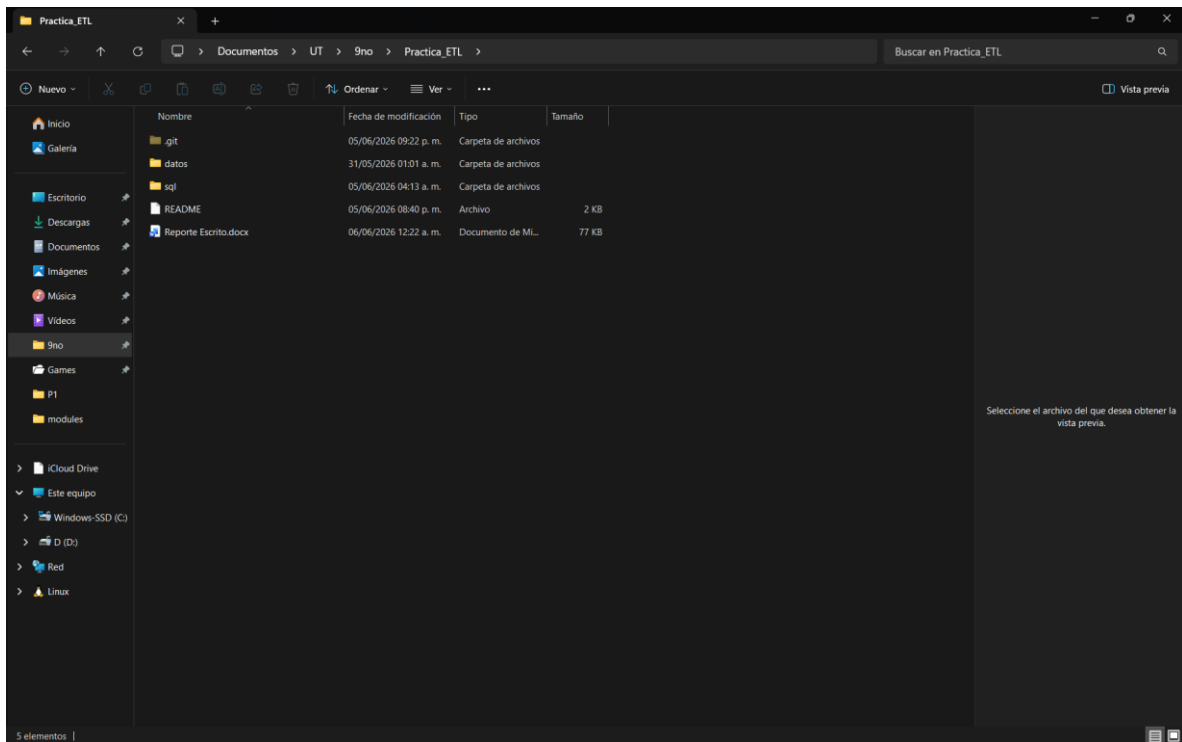


Figura 1. Estructura del proyecto y archivos utilizados para la práctica.

3.2 Ejecución de la herramienta CSV Scanner

Posteriormente se ejecutó la herramienta CSV Scanner, desarrollada como apoyo para el análisis del archivo CSV. Esta herramienta permitió cargar el dataset, revisar su estructura general, identificar características de calidad de datos y generar recursos útiles para las etapas posteriores del proceso.



Figuras 2. Ejecución de la herramienta CSV Scanner mediante Streamlit.

3.3 Carga y análisis del dataset Global Superstore Sales

En esta etapa se cargó el archivo correspondiente al dataset **Global Superstore Sales** dentro de la herramienta. A partir de esta carga se obtuvo información general sobre el conjunto de datos, como cantidad de registros, número de columnas, estructura general y uso de memoria.

Carga de Archivo CSV

Selecciona un archivo CSV

Drag and drop file here
Limit 200MB per file • CSV

Browse files

Tamaño: **10552.86 KB**

SuperStoreOrders.csv 10.3MB

☒ Archivo cargado correctamente

Consejos de Uso:

- Carga un archivo CSV pequeño primero para probar
- Si el análisis es lento, el archivo puede ser muy grande
- Revisa todas las pestañas del reporte

Filas: **51,290** Columnas: **21** Delimitador: **'** Modo: **permissive**

Vista Previa de Datos

	order_id	order_date	ship_date	ship_mode	customer_name	segment	state	country	market	region	product_id
0	AG-2011-2040	1/1/2011	6/1/2011	Standard Class	Toby Braunhardt	Consumer	Constantine	Algeria	Africa	Africa	OFF-TEN-1
1	IN-2011-47883	1/1/2011	8/1/2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	OFF-SU-10
2	HU-2011-1220	1/1/2011	5/1/2011	Second Class	Annie Thurman	Consumer	Budapest	Hungary	EMEA	EMEA	OFF-TEN-1
3	IT-2011-3647632	1/1/2011	5/1/2011	Second Class	Eugene Moren	Home Office	Stockholm	Sweden	EU	North	OFF-PA-10x
4	IN-2011-47883	1/1/2011	8/1/2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	FUR-FU-10
5	IN-2011-47883	1/1/2011	8/1/2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	OFF-PA-10x

Figura 3. Carga del archivo CSV y vista previa de los datos.

3.4 Resultados del análisis de calidad de datos

Una vez cargado el archivo, la herramienta generó un análisis de calidad de datos. En esta sección se evidencian métricas relacionadas con valores nulos, duplicados, columnas problemáticas, tipos de datos y otros aspectos relevantes para determinar el estado inicial del dataset.

Configuración

Opciones de Carga

Consejos de Uso:

- Carga un archivo CSV pequeño primero para probar
- Si el análisis es lento, el archivo puede ser muy grande
- Revisa todas las pestañas del reporte

Analizador de Calidad de Datos CSV

Herramienta local para analizar archivos CSV e identificar posibles problemas de calidad de datos que requieren validación posterior.

Importante: Este análisis es orientativo y preliminar. Los resultados no reemplazan validaciones formales de ETL ni controles de base de datos. Revisa con herramientas especializadas antes de tomar decisiones críticas.

Cargar ArchivoAnálisisValidacionesRecomendacionesDatos ProblemáticosCódigo SQL/PythonConfiguraciónDescargar Reporte

Análisis de Calidad de Datos

Información General

Total de Filas	Total de Columnas	Total de Celdas	Memoria Usada
51290	21	1077090	48.77 MB

Análisis de Valores Nulos

Celdas Nulas Totales	% Nulos Global	Columnas Problemáticas
0	0.00%	0

Análisis de Duplicados

Filas Duplicadas	% Duplicados
0	0.00%

Configuración

Opciones de Carga

Consejos de Uso:

- Carga un archivo CSV pequeño primero para probar
- Si el análisis es lento, el archivo puede ser muy grande
- Revisa todas las pestañas del reporte

Validaciones de Datos

Validación por Dominio (Opcional)

Selecciona un dominio para validar tus datos contra reglas predefinidas. Esto es referencia, no validación formal. Requiere revisión posterior.

Selecciona un dominio:

ECOMMERCE - E-Commerce y Retail

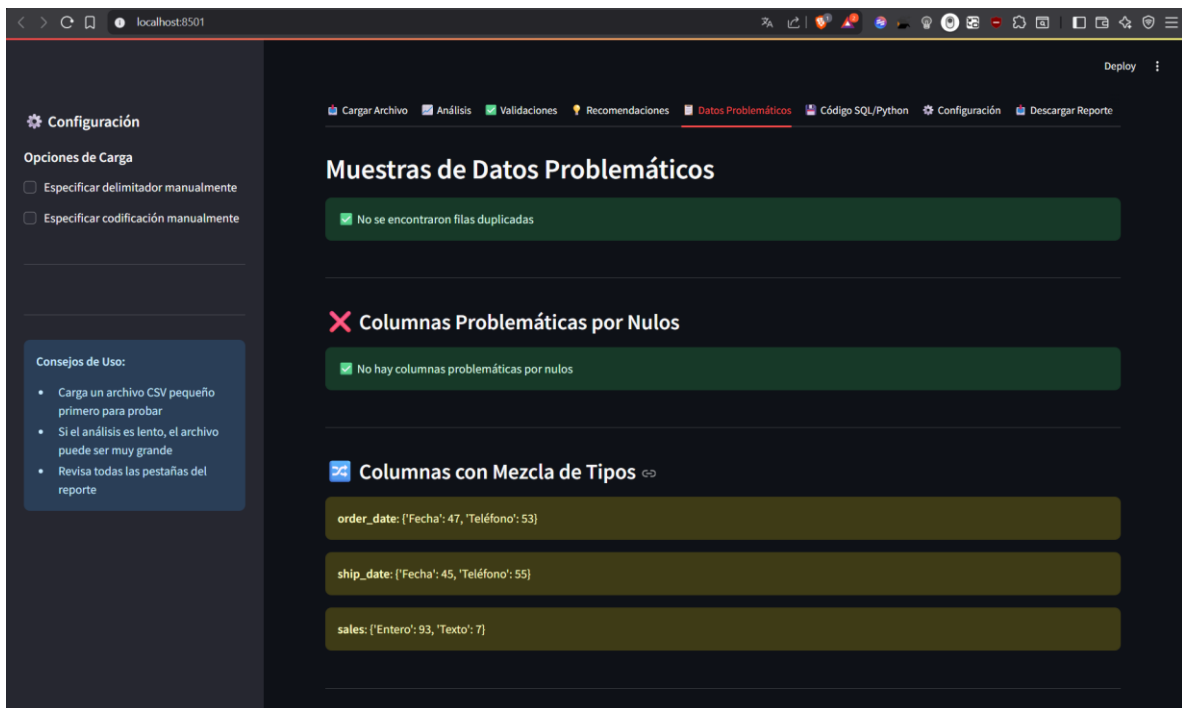
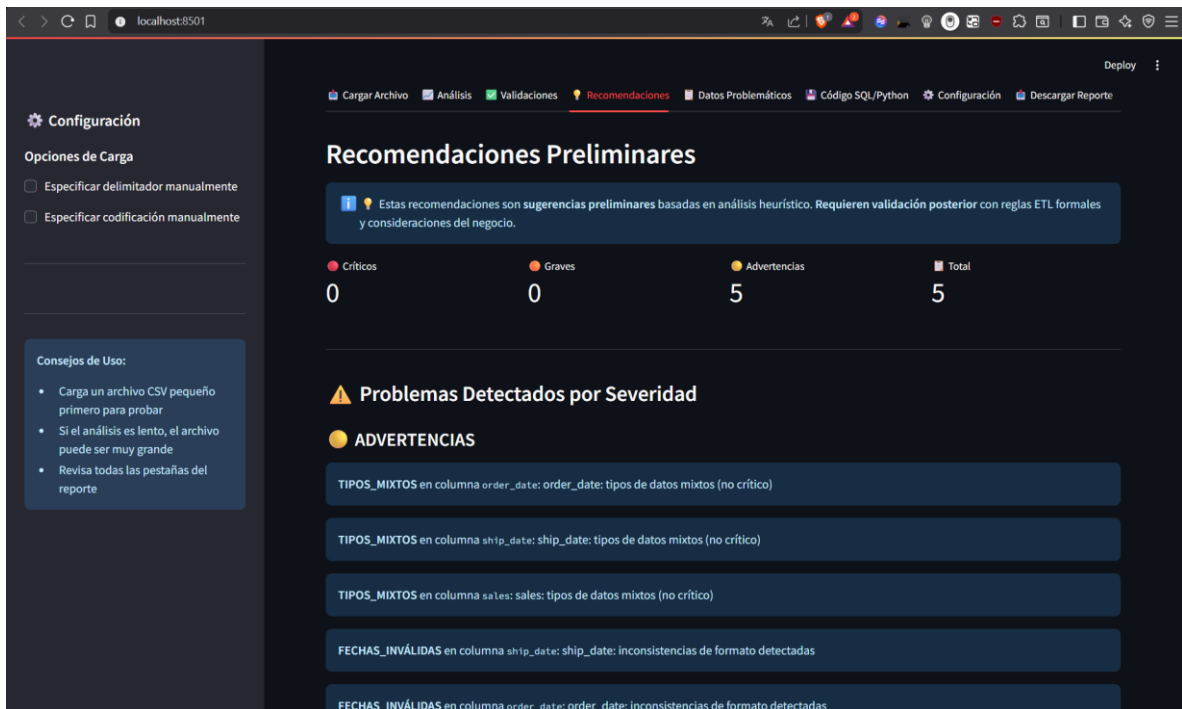
No se encontraron columnas coincidentes para dominio 'ecommerce'

Validación de Fechas

ship_date - Válidas	ship_date - Inválidas	ship_date - Estado
51290	0	PROBLEMA
order_date - Válidas	order_date - Inválidas	order_date - Estado
51290	0	PROBLEMA

Validación Numérica

quantity - Mín	quantity - Máx
1.00	14.00
discount - Mín	discount - Máx



Figuras 4, 4.1, 4.2 y 4.3. Resultados del análisis general de calidad de datos.

3.5 Generación de scripts y reportes

Como parte del proceso de transformación, la herramienta generó scripts de apoyo para la limpieza y carga de datos, incluyendo código SQL, código Python, configuración complementaria y reportes generales del análisis. Estos archivos sirvieron como base para construir y ajustar el proceso final utilizado en PostgreSQL.

The screenshot shows the 'Código Generado para BD y Limpieza' section. The left sidebar contains 'Configuración' and 'Consejos de Uso'. The main area has tabs for 'Cargar Archivo', 'Análisis', 'Validaciones', 'Recomendaciones', 'Datos Problemáticos', 'Código SQL/Python', 'Configuración', and 'Descargar Reporte'. The 'Código SQL/Python' tab is active, displaying 'Configuración del Script de Base de Datos'. It includes fields for 'Selección el motor de Base de Datos objetivo:' (PostgreSQL), 'Nombre de la tabla limpia:' (datos_limpios), and 'Nombre de la tabla origen:' (staging.orders). Below these, it shows the generated SQL code for creating tables and indexes, and the Python code for cleaning the data using pandas and numpy.

Configuración

Opciones de Carga

- ☐ Especificar delimitador manualmente
- ☐ Especificar codificación manualmente

Consejos de Uso:

- Carga un archivo CSV pequeño primero para probar
- Si el análisis es lento, el archivo puede ser muy grande
- Revisa todas las pestañas del reporte

Código Generado para BD y Limpieza

Configuración del Script de Base de Datos

Selección el motor de Base de Datos objetivo: PostgreSQL

Nombre de la tabla limpia: datos_limpios

Nombre de la tabla origen: staging.orders

Dialecto seleccionado: PostgreSQL - Tabla limpia: datos_limpios - Tabla origen: staging.orders

SQL Completo

```
-- =====
-- Script SQL generado para limpieza y
-- Generado: 05/06/2026 02:34:49
-- Dialecto: PostgreSQL
-- =====

-- 1. Crear tabla limpia
-- =====
CREATE TABLE IF NOT EXISTS "datos_limpios"
("id_registro" BIGINT GENERATED ALWAYS AS IDENTITY,
"order_id" VARCHAR(255),
"order_date" DATE,
"ship_date" DATE,
"ship_mode" VARCHAR(255));

CREATE INDEX IF NOT EXISTS "idx_datos_limpios_order_id"
ON "datos_limpios" ("order_id");

-- 2. Crear tabla de cuarentena
-- =====
CREATE TABLE IF NOT EXISTS "datos_cuarentena"
("id_registro" BIGINT GENERATED ALWAYS AS IDENTITY,
"order_id" VARCHAR(255),
"order_date" DATE,
"ship_date" DATE,
"ship_mode" VARCHAR(255));
```

Código Python

```
# Código Python para limpiar datos antes de cargarlos a la BD
import pandas as pd
import numpy as np
from datetime import datetime

def cleanup_data(df):
    """Limpia el DataFrame antes de cargarlo a la BD"""
    df_clean = df.copy()

    # 3. Limpiar espacios en blanco
    if 'order_id' in df_clean.columns:
        df_clean['order_id'] = df_clean['order_id'].str.strip()
    if 'order_date' in df_clean.columns:
        df_clean['order_date'] = df_clean['order_date'].str.strip()
    if 'ship_date' in df_clean.columns:
        df_clean['ship_date'] = df_clean['ship_date'].str.strip()
    if 'ship_mode' in df_clean.columns:
        df_clean['ship_mode'] = df_clean['ship_mode'].str.strip()

    return df_clean
```

Configuración YAML

```
# Configuración personalizada de validación
# Generada: 05/06/2026 02:34:50

database:
  table_name: datos_limpios
  dialect: PostgreSQL
  source_table: source_table
  quarantine_table: datos_cuarentena

columns:
  order_id:
    type: Text
    nullable: False
    unique: False
  order_date:
    type: Date
    nullable: False
    unique: False
```

The screenshot shows the 'Configuración Personalizada de Validación' section. The left sidebar is the same as the previous screenshot. The main area has the same tabs, with 'Configuración' now active. It displays a message to 'Carga tu propio archivo de configuración (YAML o JSON) para personalizar las reglas de validación según tu caso específico.' Below this, there are two options: 'Cargar Configuración' (with a file upload button) and 'Plantilla de Configuración' (with a template button). The 'Plantilla de Configuración' is selected, showing a sample YAML configuration for validation rules and cleaning options.

Configuración Personalizada de Validación

Carga tu propio archivo de configuración (YAML o JSON) para personalizar las reglas de validación según tu caso específico.

Cargar Configuración

Selecciona archivo de configuración (YAML o JSON)

Drag and drop file here
Limit 200MB per file • YAML, JSON

Browse files

Plantilla de Configuración

```
# Configuración personalizada de validación

database:
  table_name: datos_limpios
  charset: utf8mb4

columns:
  nombre_columna:
    type: TEXT # TEXT, INT, DECIMAL, DATE
    nullable: false
    unique: false
    validations: [email] # email, phone, date

cleaning:
  trim_whitespace: true
  remove_duplicates: true
  handle_nulls: remove # remove, fill_default, keep
  standardize_case: true

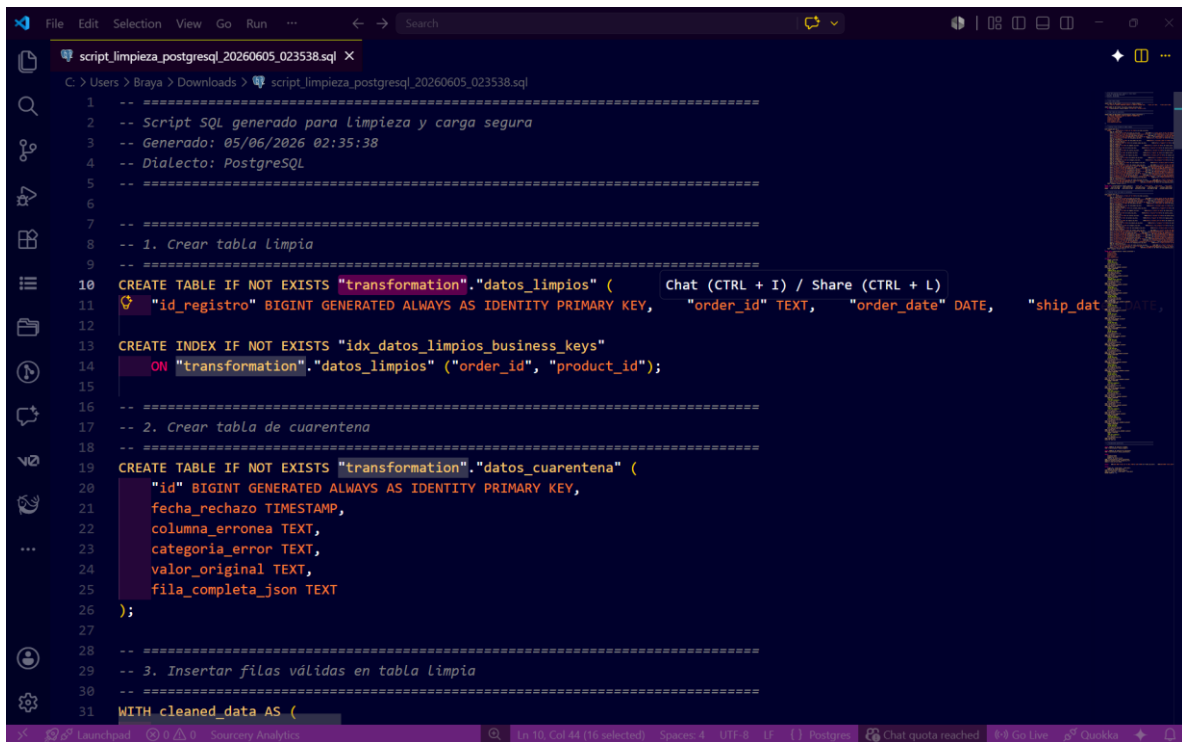
validation_rules:
  min_rows: 10
  max_null_percent: 30
  email_valid: true
  phone_valid: true
```



Figuras 5, 5.1 y 5.2. Generación de scripts SQL, Python, configuración YAML y generación de reportes.

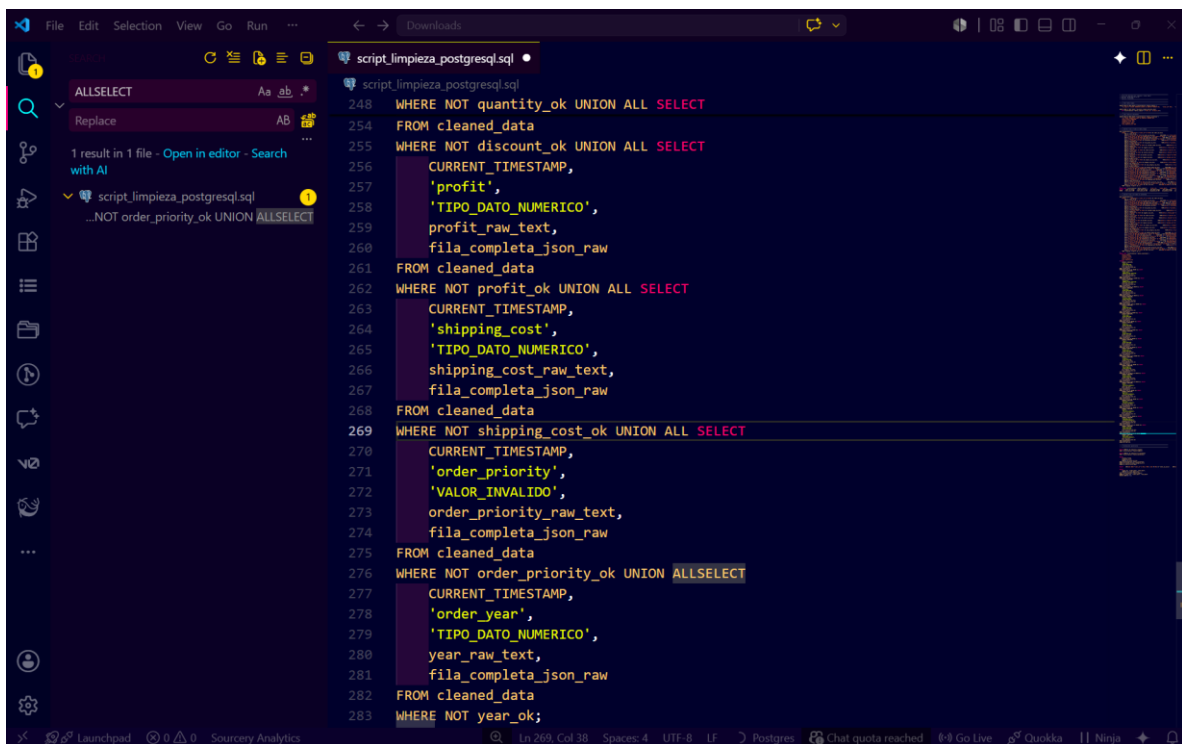
3.6 Revisión y corrección del script SQL

Después de generar el script SQL, se realizó una revisión manual para corregir detalles específicos, como nombres de campos, nombres de tablas, esquema de destino y adecuación del script al entorno PostgreSQL utilizado. Esta etapa fue necesaria para garantizar que el script pudiera ejecutarse correctamente en el entorno real de la práctica.



The screenshot shows a code editor with a file named `script_limpeza_postgresql_20260605_023538.sql`. The script is a PostgreSQL SQL file for data cleaning and loading. It includes comments in Spanish and SQL code for creating tables and indexes. The code is as follows:

```
1  -- =====
2  -- Script SQL generado para limpieza y carga segura
3  -- Generado: 05/06/2026 02:35:38
4  -- Dialecto: PostgreSQL
5  -- =====
6
7  -- =====
8  -- 1. Crear tabla Limpia
9  -- =====
10 CREATE TABLE IF NOT EXISTS "transformation"."datos_limpios" (
11     "id_registro" BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
12     "order_id" TEXT,
13     "order_date" DATE,
14     "ship_date" DATE,
15     "product_id" TEXT,
16     "quantity" TEXT,
17     "discount" TEXT,
18     "profit" TEXT,
19     "shipping_cost" TEXT,
20     "order_priority" TEXT,
21     "year" TEXT,
22     "file_raw_text" TEXT,
23     "file_json_raw_text" TEXT,
24     "file_json_raw" TEXT,
25     "file_json" TEXT
26 );
27
28 -- =====
29 -- 2. Crear tabla de cuarentena
30 -- =====
31 CREATE TABLE IF NOT EXISTS "transformation"."datos_cuarentena" (
32     "id" BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
33     "fecha_rechazo" TIMESTAMP,
34     "columna_erronea" TEXT,
35     "categoria_error" TEXT,
36     "valor_original" TEXT,
37     "fila_completa_json" TEXT
38 );
39
40 -- =====
41 -- 3. Insertar filas válidas en tabla Limpia
42 -- =====
43 WITH cleaned_data AS (
44     SELECT
45         "id_registro",
46         "order_id",
47         "order_date",
48         "ship_date",
49         "product_id",
50         "quantity",
51         "discount",
52         "profit",
53         "shipping_cost",
54         "order_priority",
55         "year",
56         "file_raw_text",
57         "file_json_raw_text",
58         "file_json_raw",
59         "file_json"
60     FROM "transformation"."datos_limpios"
61     WHERE NOT quantity_ok UNION ALL SELECT
62         "id_registro",
63         "order_id",
64         "order_date",
65         "ship_date",
66         "product_id",
67         "quantity",
68         "discount",
69         "profit",
70         "shipping_cost",
71         "order_priority",
72         "year",
73         "file_raw_text",
74         "file_json_raw_text",
75         "file_json_raw",
76         "file_json"
77     FROM "transformation"."datos_limpios"
78     WHERE NOT discount_ok UNION ALL SELECT
79         "id_registro",
80         "order_id",
81         "order_date",
82         "ship_date",
83         "product_id",
84         "quantity",
85         "discount",
86         "profit",
87         "shipping_cost",
88         "order_priority",
89         "year",
90         "file_raw_text",
91         "file_json_raw_text",
92         "file_json_raw",
93         "file_json"
94     FROM "transformation"."datos_limpios"
95     WHERE NOT profit_ok UNION ALL SELECT
96         "id_registro",
97         "order_id",
98         "order_date",
99         "ship_date",
100        "product_id",
101        "quantity",
102        "discount",
103        "profit",
104        "shipping_cost",
105        "order_priority",
106        "year",
107        "file_raw_text",
108        "file_json_raw_text",
109        "file_json_raw",
110        "file_json"
111    FROM "transformation"."datos_limpios"
112    WHERE NOT shipping_cost_ok UNION ALL SELECT
113        "id_registro",
114        "order_id",
115        "order_date",
116        "ship_date",
117        "product_id",
118        "quantity",
119        "discount",
120        "profit",
121        "shipping_cost",
122        "order_priority",
123        "year",
124        "file_raw_text",
125        "file_json_raw_text",
126        "file_json_raw",
127        "file_json"
128    FROM "transformation"."datos_limpios"
129    WHERE NOT order_priority_ok UNION ALL SELECT
130        "id_registro",
131        "order_id",
132        "order_date",
133        "ship_date",
134        "product_id",
135        "quantity",
136        "discount",
137        "profit",
138        "shipping_cost",
139        "order_priority",
140        "year",
141        "file_raw_text",
142        "file_json_raw_text",
143        "file_json_raw",
144        "file_json"
145    FROM "transformation"."datos_limpios"
146    WHERE NOT year_ok;
147
```



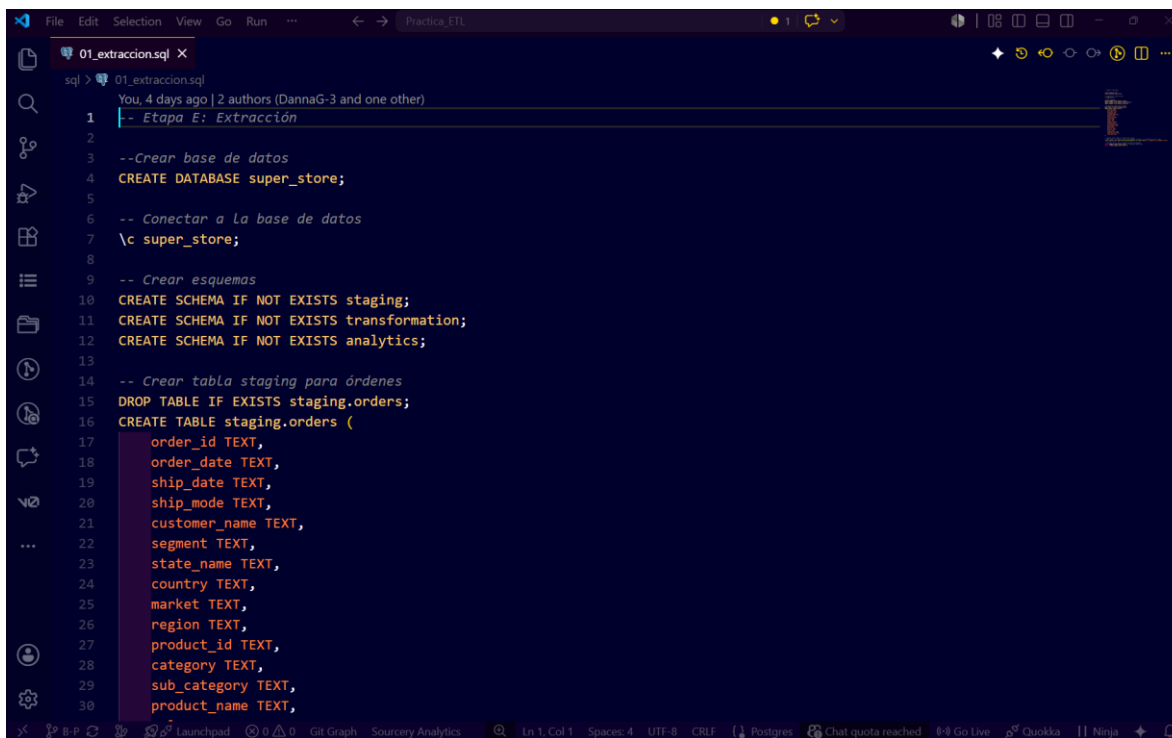
The screenshot shows the same code editor with the corrected SQL script. The script is a PostgreSQL SQL file for data cleaning and loading. It includes comments in Spanish and SQL code for creating tables and indexes. The code is as follows:

```
248 WHERE NOT quantity_ok UNION ALL SELECT
249 FROM cleaned_data
250 WHERE NOT discount_ok UNION ALL SELECT
251 CURRENT_TIMESTAMP,
252 'profit',
253 'TIPO_DATO_NUMERICO',
254 profit_raw_text,
255 fila_completa_json_raw
256 FROM cleaned_data
257 WHERE NOT profit_ok UNION ALL SELECT
258 CURRENT_TIMESTAMP,
259 'shipping_cost',
260 'TIPO_DATO_NUMERICO',
261 shipping_cost_raw_text,
262 fila_completa_json_raw
263 FROM cleaned_data
264 WHERE NOT shipping_cost_ok UNION ALL SELECT
265 CURRENT_TIMESTAMP,
266 'order_priority',
267 'VALOR_INVALIDO',
268 order_priority_raw_text,
269 fila_completa_json_raw
270 FROM cleaned_data
271 WHERE NOT order_priority_ok UNION ALL SELECT
272 CURRENT_TIMESTAMP,
273 'order_year',
274 'TIPO_DATO_NUMERICO',
275 year_raw_text,
276 fila_completa_json_raw
277 FROM cleaned_data
278 WHERE NOT year_ok;
```

Figuras 6 y 6.1. Corrección y ajuste del script SQL generado.

3.7 Ejecución de la extracción en PostgreSQL

Posteriormente se ejecutó el archivo **01_extraccion.sql**, utilizando consola de comandos y el comando **\copy** para cargar los datos originales en PostgreSQL. Como resultado se creó la base de datos de trabajo, el esquema **staging** y la tabla **orders**, conservando los datos originales en bruto.



```
1  -- Etapa E: Extracción
2
3  --Crear base de datos
4  CREATE DATABASE super_store;
5
6  -- Conectar a La base de datos
7  \c super_store;
8
9  -- Crear esquemas
10 CREATE SCHEMA IF NOT EXISTS staging;
11 CREATE SCHEMA IF NOT EXISTS transformation;
12 CREATE SCHEMA IF NOT EXISTS analytics;
13
14 -- Crear tabla staging para órdenes
15 DROP TABLE IF EXISTS staging.orders;
16 CREATE TABLE staging.orders (
17     order_id TEXT,
18     order_date TEXT,
19     ship_date TEXT,
20     ship_mode TEXT,
21     customer_name TEXT,
22     segment TEXT,
23     state_name TEXT,
24     country TEXT,
25     market TEXT,
26     region TEXT,
27     product_id TEXT,
28     category TEXT,
29     sub_category TEXT,
30     product_name TEXT,
```

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.26200.8524]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Braya\Documents\UT\9no\Practica_ETL\sql>"C:\Program Files\PostgreSQL\18\bin\psql.exe" -h localhost -p 5433 -U postgres -d super_store -v ON_ERROR_S
TOP=1 -f "C:\Users\Braya\Documents\UT\9no\Practica_ETL\sql\01_extraccion.sql"
Contraseña para usuario postgres: |
```

pgAdmin 4

Object Explorer

- Publications
- Schemas (4)
 - analytics
 - public
 - staging
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (1)
 - orders
 - Trigger Functions
 - Types
 - Views
 - transformation
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures

super_store/postgres@PostgreSQL 18

Query

```
select * from staging.orders
```

Data Output

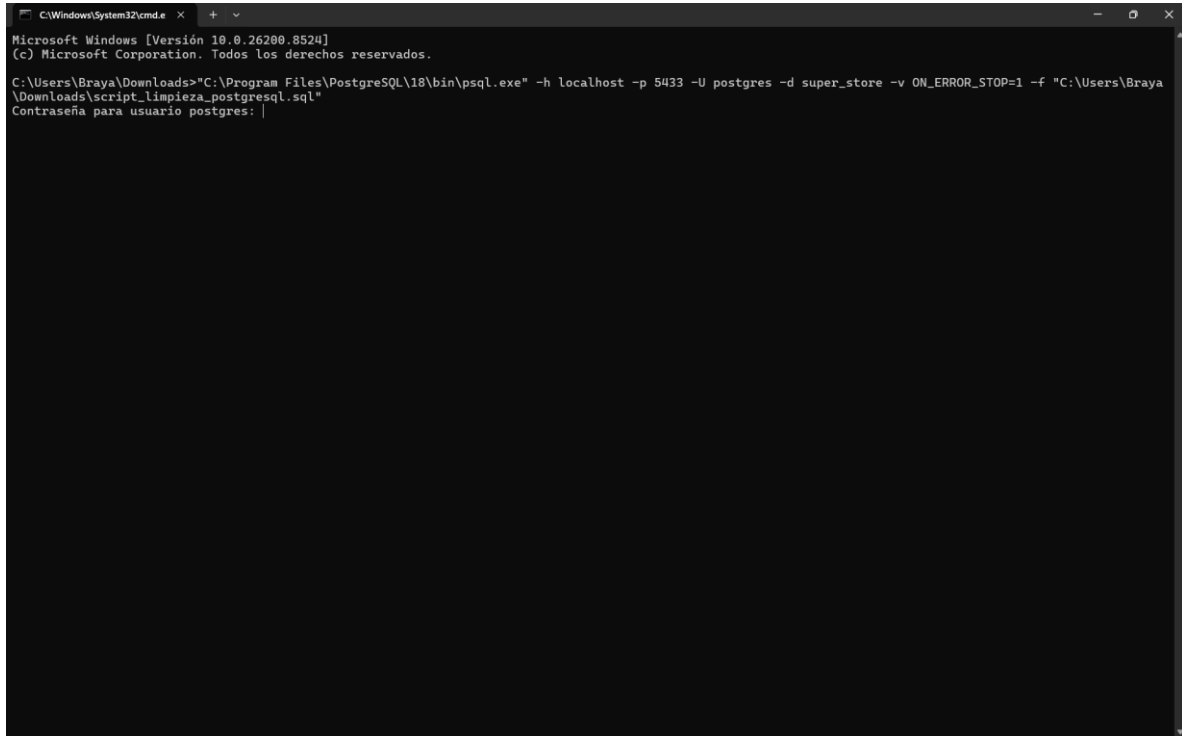
	order_id	order_date	ship_date	ship_mode	customer_name	segment	state_name	country	market	region
	text	text	text	text	text	text	text	text	text	text
1	AG-2011-2040	1/1/2011	6/1/2011	Standard Cla...	Toby Brauhardt	Consumer	Constantine	Algeria	Africa	Africa
2	IN-2011-47883	1/1/2011	8/1/2011	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania
3	HU-2011-1220	1/1/2011	5/1/2011	Second Class	Annie Thurman	Consumer	Budapest	Hungary	EMEA	EMEA
4	IT-2011-3647632	1/1/2011	5/1/2011	Second Class	Eugene Moren	Home Offi...	Stockholm	Sweden	EU	North
5	IN-2011-47883	1/1/2011	8/1/2011	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania
6	IN-2011-47883	1/1/2011	8/1/2011	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania
7	CA-2011-1510	2/1/2011	6/1/2011	Standard Cla...	Magdelene Morse	Consumer	Ontario	Canada	Canada	Canada
8	IN-2011-79397	3/1/2011	3/1/2011	Same Day	Kean Nguyen	Corporate	New South Wales	Australia	APAC	Oceania
9	ID-2011-80230	3/1/2011	9/1/2011	Standard Cla...	Ken Lonsdale	Consumer	Auckland	New Zealand	APAC	Oceania
10	IZ-2011-4680	3/1/2011	7/1/2011	Standard Cla...	Lindsay Williams	Corporate	Ninawa	Iraq	EMEA	EMEA
11	IN-2011-65159	3/1/2011	7/1/2011	Second Class	Larry Blacks	Consumer	National Capital	Philippines	APAC	Southeast A...
12	IN-2011-65159	3/1/2011	7/1/2011	Second Class	Larry Blacks	Consumer	National Capital	Philippines	APAC	Southeast A...
13	ES-2011-48696...	3/1/2011	7/1/2011	Standard Cla...	Dorothy Dickinson	Consumer	England	United Kingdom	EU	North
14	IN-2011-33652	3/1/2011	9/1/2011	Standard Cla...	Dennis Pardue	Home Offi...	Sarawak	Malaysia	APAC	Southeast A...
15	ID-2011-80230	3/1/2011	9/1/2011	Standard Cla...	Ken Lonsdale	Consumer	Auckland	New Zealand	APAC	Oceania

Total rows: 51290 Query complete 00:00:00.353

Figuras 7, 7.1 y 7.2. Ejecución del script de extracción y creación de la tabla staging.orders.

3.8 Transformación hacia el esquema transformation

Una vez cargados los datos en **staging**, se ejecutó el script correspondiente a la transformación para crear las estructuras procesadas dentro del esquema **transformation**. En esta fase se generó la tabla **datos_limpios**, con los datos estandarizados y preparados para análisis, así como la tabla **datos_cuarentena**, destinada a almacenar registros inválidos en caso de ser detectados.



```
C:\Windows\System32\cmd.exe X + -
Microsoft Windows [Versión 10.0.26200.8524]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Braya\Downloads>"C:\Program Files\PostgreSQL\18\bin\psql.exe" -h localhost -p 5433 -U postgres -d super_store -v ON_ERROR_STOP=1 -f "C:\Users\Braya\Downloads\script_limpieza_postgresql.sql"
Contraseña para usuario postgres: |
```

```
C:\Windows\System32\cmd.e  X  +  v

ID-2012-55905 | 2012-08-14 | 2012-08-16 | 2
IT-2014-2725928 | 2014-06-16 | 2014-06-19 | 2
ES-2014-5801819 | 2014-03-18 | 2014-03-22 | 2
MX-2014-116176 | 2014-07-08 | 2014-07-14 | 2
CA-2012-148376 | 2012-12-27 | 2012-12-31 | 2
IN-2012-72684 | 2012-03-27 | 2012-04-01 | 3
MX-2014-135188 | 2014-09-09 | 2014-09-13 | 3
IN-2012-16411 | 2012-09-18 | 2012-09-22 | 2
NI-2014-8510 | 2014-12-03 | 2014-12-04 | 3
CA-2011-151906 | 2011-06-04 | 2011-06-09 | 4
MX-2012-163272 | 2012-05-06 | 2012-05-18 | 3
CA-2012-157770 | 2012-12-13 | 2012-12-15 | 4
ES-2011-3717351 | 2011-11-23 | 2011-11-29 | 2
MX-2013-164938 | 2013-04-17 | 2013-04-22 | 3
IT-2012-2825037 | 2012-08-14 | 2012-08-20 | 3
IN-2013-53882 | 2013-09-19 | 2013-09-21 | 2
EZ-2013-9740 | 2013-06-19 | 2013-06-24 | 4
MX-2012-133823 | 2012-05-24 | 2012-05-28 | 2
IT-2014-0652298 | 2014-04-22 | 2014-04-23 | 2
KG-2014-2870 | 2014-12-02 | 2014-12-05 | 3
CA-2014-152485 | 2014-09-05 | 2014-09-09 | 3
IT-2014-2866257 | 2014-08-04 | 2014-08-08 | 3
CA-2013-159212 | 2013-11-02 | 2013-11-06 | 2
EG-2013-3870 | 2013-11-28 | 2013-12-02 | 2
US-2013-103051 | 2013-11-04 | 2013-11-10 | 2
MX-2013-143028 | 2013-11-28 | 2013-12-04 | 2
TU-2014-8390 | 2014-02-25 | 2014-03-03 | 5
US-2013-111563 | 2013-11-05 | 2013-11-09 | 2
IN-2014-50179 | 2014-10-13 | 2014-10-17 | 3
CA-2012-111703 | 2012-07-02 | 2012-07-09 | 2
TU-2014-8680 | 2014-08-09 | 2014-08-11 | 3
ID-2011-55667 | 2011-02-23 | 2011-02-26 | 3
CA-2013-106894 | 2013-02-07 | 2013-02-07 | 2
SG-2014-2090 | 2014-02-27 | 2014-03-05 | 4
MX-2011-132066 | 2011-07-07 | 2011-07-13 | 2
RS-2014-0620 | 2014-08-14 | 2014-08-16 | 3
IN-2011-33855 | 2011-03-08 | 2011-03-12 | 4
ES-2012-4869643 | 2012-06-01 | 2012-06-06 | 2
ES-2011-4789704 | 2011-09-22 | 2011-09-29 | 2
MX-2012-154396 | 2012-10-23 | 2012-10-25 | 2
ES-2013-4981983 | 2013-08-23 | 2013-08-25 | 2
ES-2014-4936686 | 2014-12-10 | 2014-12-12 | 3
TU-2013-420 | 2013-08-16 | 2013-08-22 | 6
MX-2012-101021 | 2012-06-07 | 2012-06-12 | 4
(12790 filas)

C:\Users\Braya\Downloads>
```

pgAdmin 4

File Object Tools Edit View Window Help

super_store/postgres@PostgreSQL 18 x

super_store/postgres@PostgreSQL 18

No limit

Query Query History

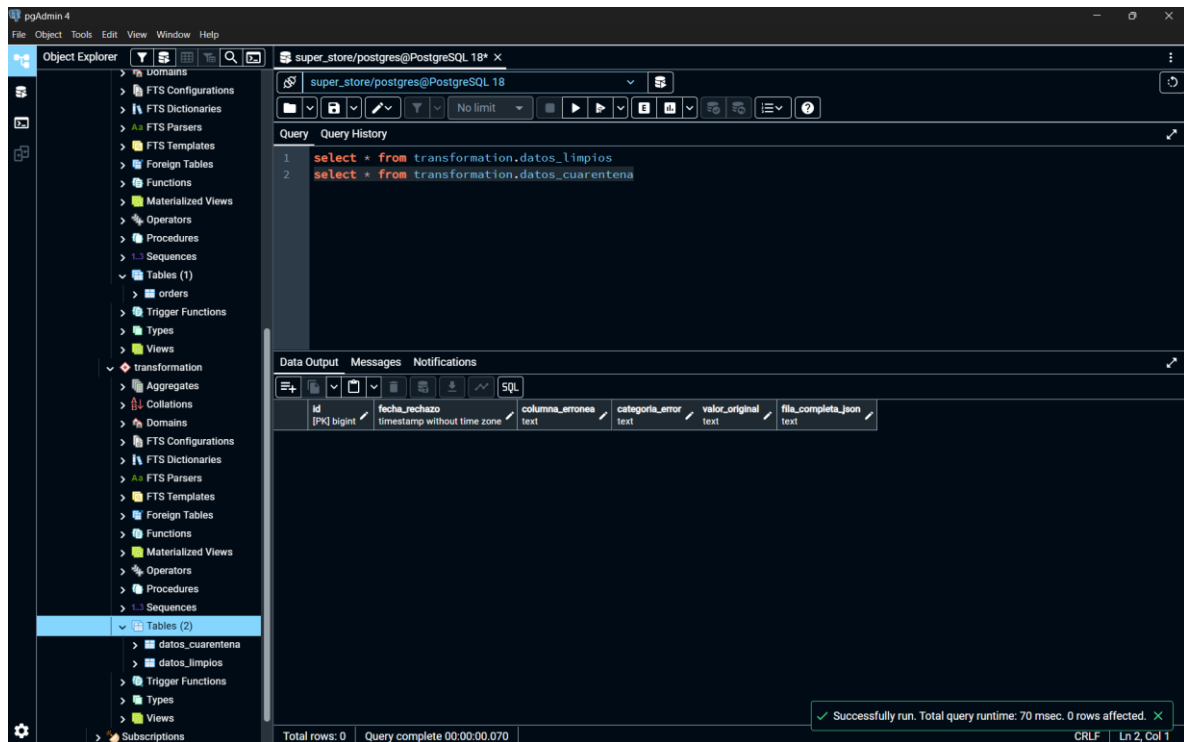
```
1 select * from transformation.datos_limpios
2 select * from transformation.datos_cuarentena
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 52

	id_registro (PK) bigint	order_id text	order_date date	ship_date date	ship_mode text	customer_name text	segment text	state_name text	country text	market text	re te
1	1	AG-2011-2040	2011-01-01	2011-01-06	Standard Cla...	Toby Braunhardt	Consumer	Constantine	Algeria	Africa	A
2	2	IN-2011-47883	2011-01-01	2011-01-08	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	O
3	3	HU-2011-1220	2011-01-01	2011-01-05	Second Class	Annie Thurman	Consumer	Budapest	Hungary	EMEA	E
4	4	IT-2011-3647632	2011-01-01	2011-01-05	Second Class	Eugene Moran	Home Offi...	Stockholm	Sweden	EU	N
5	5	IN-2011-47883	2011-01-01	2011-01-08	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	O
6	6	IN-2011-47883	2011-01-01	2011-01-08	Standard Cla...	Joseph Holt	Consumer	New South Wales	Australia	APAC	O
7	7	CA-2011-1510	2011-01-02	2011-01-06	Standard Cla...	Magdelene Morse	Consumer	Ontario	Canada	Canada	C
8	8	IN-2011-79397	2011-01-03	2011-01-03	Same Day	Kean Nguyen	Corporate	New South Wales	Australia	APAC	O
9	9	ID-2011-40230	2011-01-03	2011-01-09	Standard Cla...	Ken Lonsdale	Consumer	Auckland	New Zealand	APAC	O
10	10	IZ-2011-4680	2011-01-03	2011-01-07	Standard Cla...	Lindsay Williams	Corporate	Ninawa	Iraq	EMEA	E
11	11	IN-2011-65159	2011-01-03	2011-01-07	Second Class	Larry Blacks	Consumer	National Capital	Philippines	APAC	S
12	12	IN-2011-65159	2011-01-03	2011-01-07	Second Class	Larry Blacks	Consumer	National Capital	Philippines	APAC	S
13	13	ES-2011-48696...	2011-01-03	2011-01-07	Standard Cla...	Dorothy Dickinson	Consumer	England	United Kingdom	EU	N
14	14	IN-2011-33652	2011-01-03	2011-01-09	Standard Cla...	Dennis Pardue	Home Offi...	Sarawak	Malaysia	APAC	S
15	15	ID-2011-40230	2011-01-03	2011-01-09	Standard Cla...	Ken Lonsdale	Consumer	Auckland	New Zealand	APAC	O

Total rows: 51290 Query complete 00:00:00.457 CRLF Ln 1, Col 1



Figuras 8, 8.1, 8.2 y 8.3. Creación y carga de las tablas transformation.datos_limpios y transformation.datos_cuarentena.

3.9 Validación final mediante pgAdmin

Finalmente, se realizaron consultas de validación mediante la herramienta **Query Tool** de **pgAdmin**. Estas consultas permitieron revisar conteos, estructura de tablas, valores nulos, duplicados, consistencia de fechas, rangos numéricos y estado de la tabla de cuarentena. Los resultados confirmaron que los datos procesados fueron cargados correctamente y que no existieron registros corruptos que requirieran aislamiento.

The screenshot shows a code editor with a file explorer on the left and a script editor on the right. The file explorer shows a directory structure with files like 'script_validación_postgresql.sql' and 'script_limpieza_postgresql.sql'. The script editor contains the following SQL code:

```
1  -- Script de validación de datos
2  -- Generado: 05/06/2026 04:32:35
3  -- Dialecto: PostgreSQL
4  -- Tablas:
5  --   transformation.datos_limpios
6  --   transformation.datos_cuarentena
7  --
8  -- =====
9
10 -- 1. Validar existencia de tablas
11 -- =====
12
13 SELECT
14     table_schema,
15     table_name
16 FROM information_schema.tables
17 WHERE table_schema = 'transformation'
18    AND table_name IN ('datos_limpios', 'datos_cuarentena')
19 ORDER BY table_name;
20
21 -- =====
22 -- 2. Conteo general de registros
23 -- =====
24
25 SELECT
26     'datos_limpios' AS tabla,
27     COUNT(*) AS total_registros
28 FROM transformation.datos_limpios
29 UNION ALL
30 SELECT
31     'datos_cuarentena' AS tabla,
32     COUNT(*) AS total_registros
```

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'transformation' schema and its tables. The right pane shows the execution of the script, with the query history and the data output table.

Query History:

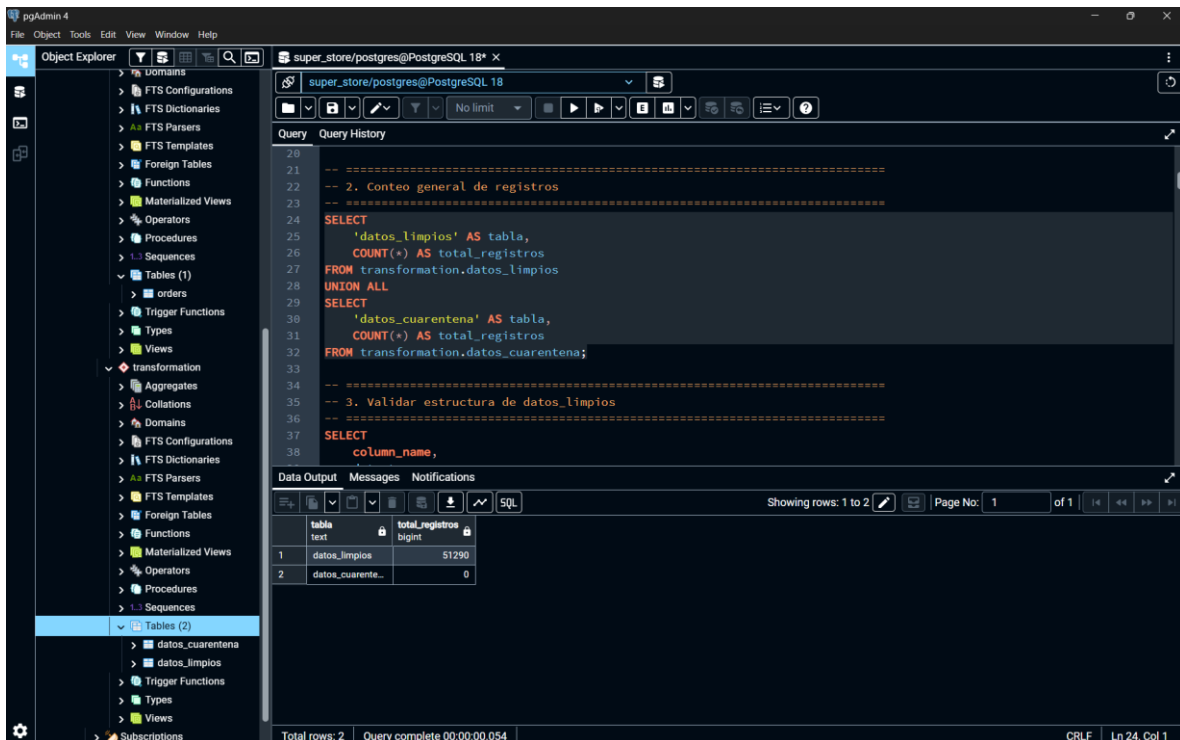
```
2  -- Script de validación de datos
3  -- Generado: 05/06/2026 04:32:35
4  -- Dialecto: PostgreSQL
5  -- Tablas:
6  --   transformation.datos_limpios
7  --   transformation.datos_cuarentena
8  --
9  -- =====
10 -- 1. Validar existencia de tablas
11 -- =====
12
13 SELECT
14     table_schema,
15     table_name
16 FROM information_schema.tables
17 WHERE table_schema = 'transformation'
18    AND table_name IN ('datos_limpios', 'datos_cuarentena')
19 ORDER BY table_name;
20
```

Data Output:

table_schema	table_name
transformation	datos_cuarentena
transformation	datos_limpios

Messages:

Successfully run. Total query runtime: 71 msec. 2 rows affected.



Figuras 9, 9.1 y 9.2. Validaciones finales ejecutadas sobre las tablas procesadas en PostgreSQL.

Nota: Las validaciones completas están en el script “03_validacion.sql” y en el documento de evidencias anexo*

4. Base de Datos Procesada

La base de datos utilizada para la práctica (super_store) fue diseñada bajo una estructura separada por esquemas, con el objetivo de diferenciar claramente los datos originales de los datos procesados. Esta organización permitió mantener trazabilidad durante el proceso y evitar modificaciones directas sobre la información en bruto.

La base de datos se organizó principalmente en dos esquemas: **staging** y **transformation**. El esquema **staging** se utilizó como zona de recepción de los datos originales, mientras que el esquema **transformation** se destinó al almacenamiento de la información procesada, limpia y validada.

4.1 Esquema staging

El esquema **staging** representa la primera capa de almacenamiento dentro de la base de datos. Su función principal fue recibir los datos originales provenientes del archivo CSV del dataset **Global Superstore Sales**, sin aplicar transformaciones, validaciones o cambios en su estructura inicial.

Dentro de este esquema se creó la tabla **staging.orders**, la cual almacenó la información de las órdenes contenidas en el dataset. En esta etapa, los campos fueron definidos como texto con el propósito de permitir la carga completa de los datos sin rechazar registros por diferencias de formato, errores de tipo o inconsistencias propias del archivo de origen.

El uso del esquema **staging** permitió conservar una copia íntegra de los datos originales, lo cual resulta importante para mantener trazabilidad y facilitar la repetición del proceso en caso de que se requiriera corregir o ajustar la transformación.

4.2 Esquema transformation

El esquema **transformation** corresponde a la capa donde se almacenaron los datos después del proceso de limpieza, estandarización y validación. A diferencia del esquema **staging**, esta capa contiene información preparada para su consulta y análisis posterior.

En este esquema se generaron dos tablas principales: **transformation.datos_limpios** y **transformation.datos_cuarentena**.

La tabla **transformation.datos_limpios** almacena los registros que fueron procesados correctamente durante la transformación. En esta tabla se aplicaron ajustes como la conversión de campos de fecha, estandarización de valores numéricos y conservación de los campos comerciales relevantes del dataset. Esta tabla representa la versión analítica del conjunto de datos, lista para ser utilizada en consultas, reportes o análisis posteriores.

Por otro lado, la tabla **transformation.datos_cuarentena** fue creada como mecanismo de control para almacenar registros que presentaran errores de formato, valores incompatibles o inconsistencias que impidieran su carga en la tabla limpia. Aunque durante la validación final no se identificaron registros corruptos que requirieran aislamiento, la existencia de esta tabla fortalece el diseño del proceso, ya que permite conservar evidencia de posibles errores sin perder información.

En conjunto, esta separación por esquemas permitió organizar el flujo de datos de forma clara: primero se conservaron los datos originales en **staging**, posteriormente se transformaron y cargaron en **transformation.datos_limpios**, y finalmente se dejó disponible una estructura de control mediante **transformation.datos_cuarentena** para registrar cualquier anomalía detectada durante el proceso.

Conclusiones

El desarrollo de esta práctica permitió implementar de forma satisfactoria un proceso de extracción, transformación y validación de datos utilizando PostgreSQL como plataforma de almacenamiento y análisis. Aunque el dataset **Global Superstore Sales** presentó un nivel de calidad considerablemente alto y no fue necesario aislar registros en la tabla de cuarentena, el proceso permitió estandarizar correctamente la información y verificar su consistencia antes de su utilización para fines analíticos.

Uno de los resultados más relevantes del proyecto fue el desarrollo de la herramienta [CSV Scanner](#), creada específicamente para apoyar el análisis y preparación de archivos CSV. Su utilización redujo significativamente el esfuerzo requerido durante la etapa de transformación, permitiendo automatizar tareas de análisis, generación de reportes y creación de scripts de apoyo para el proceso ETL.

Durante el desarrollo también fue necesario realizar actividades de verificación, corrección y refinamiento tanto de la herramienta como de los scripts SQL generados, incluyendo la validación de nombres de campos, estructuras de tablas, entornos de ejecución y compatibilidad con PostgreSQL. Estas tareas permitieron fortalecer la confiabilidad del proceso y garantizar resultados consistentes durante las pruebas realizadas.

Finalmente, la práctica no solo permitió aplicar los conceptos estudiados sobre procesos ETL y calidad de datos, sino también desarrollar una solución reutilizable que podrá emplearse en futuros proyectos de análisis de datos, facilitando la identificación de problemas de calidad y reduciendo el tiempo necesario para preparar información proveniente de archivos CSV.

En conjunto, los resultados obtenidos demuestran la importancia de los procesos de preparación y validación de datos dentro de los proyectos de analítica, ya que la calidad de la información constituye un factor fundamental para la obtención de resultados confiables.